Mamba-3 Part 1

AUTHORS
Kevin Y. Li* (bloggin)
Aakash Lahoti*
Berlin Chen*
Caitlin Wang*
Aviv Bick
J. Zico Kolter
Tri Dao
Albert Gu

AFFILIATIONS
CMU
CMU
Princeton
Princeton
CMU
CMU
Princeton, Together AI
CMU, Cartesia AI

PUBLISHED
March 16, 2026

Contents

The Mamba-3 Model
Architecture
Empirical Results
- Language Modeling
- Retrieval Tasks

Kernels Here, There, and Everywhere
- Benchmarking Latencies
- Design Choices

Next Up

[Paper] [Code]

This series is cross-posted at tridao.me

1. Part I
2. Part II

Since the release of Mamba-2 in mid-2024, most architectures have switched from Mamba-1. Why? Mamba-2 made the bet that training efficiency was the largest bottleneck for state space models (SSMs), and thus simplified the underlying SSM mechanism to deliver $2-8\times$ faster training compared to its predecessor, leading to wider adoption.

Since then, the LLM landscape has started to shift. While pretraining is still super important, more attention has been focused on post-training and deployment, both of which are *extremely inference-heavy*. The scaling of post-training methods, especially with reinforcement learning with verifiable rewards (RLVR) for coding or math, requires huge amounts of generated rollouts, and most recently, agentic workflows, such as Codex, Claude Code, or even OpenClaw, have **pushed inference demand through the roof**.

Despite the clear, growing importance of inference, many linear architectures (including Mamba-2) were developed from a training-first perspective. To accelerate pretraining, the underlying SSM was *progressively simplified* (e.g., the diagonal transition was reduced to a scalar times identity). While this brought training speed, it left the inference step "too simple" and squarely memory-bound — the GPUs aren't brr-ing but moving memory most of the time.

In this new age of inference, we care a lot about pushing the boundaries of the quality-efficiency frontier: we want the *better* models to run *faster*.

A natural question arises:

> *What would an SSM designed with **inference** in mind look like?*

The Mamba-3 Model

What's missing? The main appeal of linear models is in their name: compute scales linearly with sequence length because of a fixed-size state. Unfortunately, there is *no free lunch*. The same **fixed state size** that enables efficient computation forces the model to compress all past information into one representation, the exact opposite of a Transformer, which stores all past information through a continuously growing state (the KV cache) — a *fundamental* difference. So, if we can't grow the state, how do we make that fixed state do more work?

We see that earlier designs simplified the recurrence and the transition matrix to make training fast. However, the change also *reduced the richness* of the dynamics and left decoding memory-bound: each token update performs very little computation relative to memory movement. This provides us with three levers we can pull: **(1)** make the recurrence itself more expressive, **(2)** use a richer transition matrix, and **(3)** add more parallel (and almost free) work inside each update.

From these insights, we improve upon Mamba-2 in three core ways that:

1. increase the expressivity of the SSM mechanism through a more general recurrence derived from our **exponential-trapezoidal discretization scheme**,
2. expand the state-tracking capabilities by modeling a **complex-valued SSM system**, and
3. improve the model's general performance with little impact on decode latency by using **multi-input, multi-output (MIMO) SSMs**, which model multiple SSMs in parallel, instead of the current single-input, single-output (SISO) SSMs.

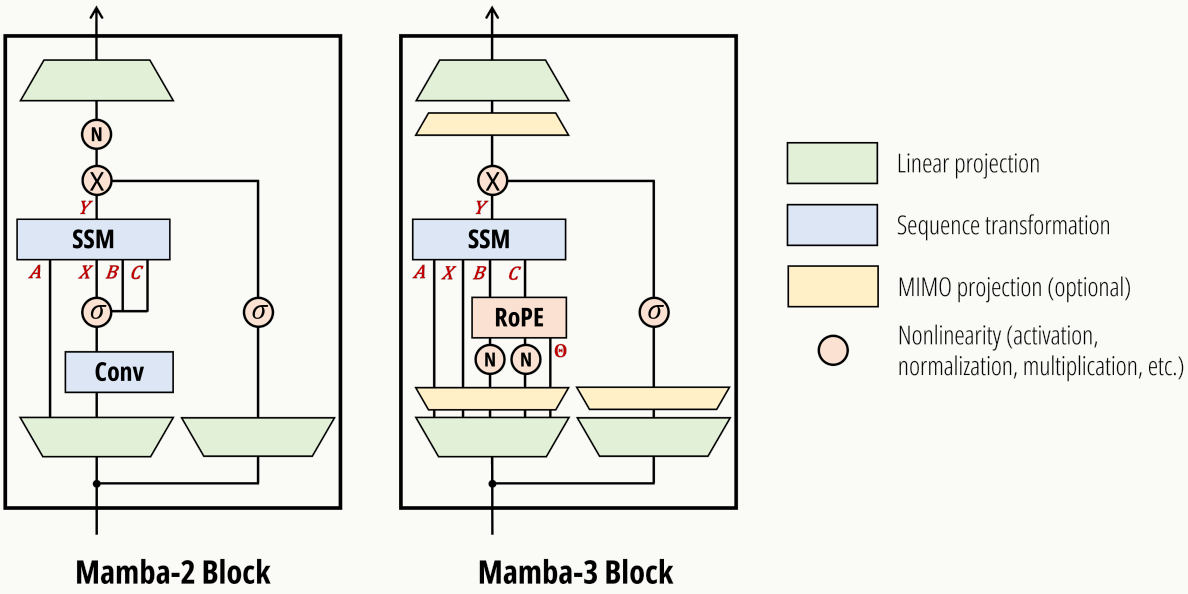
Through these three changes, **Mamba-3 pushes the frontier of performance while maintaining similar inference latency**.

> *Notably, all three of these changes are inspired by the more "classical" control theory and **state space model** literature.*

Our work goes against the grain of many modern linear architectures, which use alternative interpretations of recurrence (such as **linear attention** or **test-time training**) that *don't easily capture these concepts*.

Architecture

What has changed in the Mamba-2 layer? Beyond the three methodological upgrades to the core SSM discussed above, we've revamped the architecture a bit to make it more in line with conventional modern language models.



Mamba-3 Architecture.

Based on the diagram, you'll notice we've changed a couple of things. On a high level,

Norms. We added in QKNorm [1], which empirically stabilizes the training of Mamba-3 models. The addition of this norm brings Mamba-3 in line with contemporary Transformer and Gated DeltaNet (GDN) models. With QKNorm, the RMSNorm from Mamba-2 becomes optional. However, we empirically find that it may still be worth keeping in hybrid models due to helping length extrapolation capabilities. More on this later.

Goodbye Short Conv. We've been able to get rid of the pesky short causal convolution of Mamba-1/2 by combining (1) simple biases on B and C after BCNorm with (2) our new discretization-based recurrence. The new recurrence implicitly applies a **convolution** on the input to the hidden state, and we show how this is the case in Part 2 of our blog.

▶ Can the short conv really be removed?

▶ Quick history lesson on the short conv.

Finally, you'll notice a couple of new components, namely **RoPE** and **MIMO projections**. The RoPE module expresses complex-valued SSMs via the interpretation of complex transitions as rotations, forgoing the costly reimplementation of kernels. The MIMO projections expand the B and C matrices to the appropriate representation needed for MIMO SSMs.

We dig into the motivation and exact implementation of these two in greater detail in the second part of our blog (lots of goodies there 🎁), so for now, just think of them as **standalone, fundamental improvements** that individually contribute to improving the model's performance and/or capabilities.

Finally, our overall architecture now adopts interleaved MLP layers following the standard convention of Transformers and other linear models.

Empirical Results

We evaluate our final Mamba-3 model against other popular linear alternatives and the Transformer baseline.

Language Modeling

Model	FW-Edu ppl ↓	LAMB. ppl ↓	LAMB. acc ↑	HellaS. acc_n ↑	PIQA acc ↑	Arc-E acc ↑	Arc-C acc_n ↑	WinoGr. acc ↑	OBQA acc ↑	Average acc ↑
Transformer-180M	16.89	45.0	**32.5**	39.0	**67.1**	59.8	27.9	51.2	21.8	42.8
GDN-180M	16.52	**40.8**	31.3	40.2	66.3	**62.3**	**28.2**	51.7	22.0	43.2
Mamba-2-180M	16.76	41.8	30.9	40.1	66.8	60.1	27.3	**52.0**	**23.2**	42.9
Mamba-3-SISO-180M	**16.59**	**37.7**	**32.5**	**40.8**	66.1	61.5	27.9	**52.0**	22.8	**43.4**
Mamba-3-MIMO-180M	**16.46**	**32.1**	**34.0**	**41.0**	66.7	60.6	27.7	**52.9**	22.0	**43.5**
Transformer-440M	13.03	21.2	41.7	50.5	69.9	67.6	34.6	**56.7**	26.0	49.6
GDN-440M	13.01	**18.0**	**41.9**	50.9	70.0	67.0	34.6	56.1	**27.6**	49.7
Mamba-2-440M	13.00	19.6	40.8	**51.7**	70.6	68.8	**35.0**	54.1	26.0	49.6
Mamba-3-SISO-440M	**12.87**	19.6	40.2	**51.7**	**71.9**	**68.9**	34.4	55.8	26.0	**49.8**
Mamba-3-MIMO-440M	**12.72**	**17.1**	**43.4**	**52.8**	70.8	**69.6**	**35.6**	56.3	**28.4**	**51.0**
Transformer-880M	11.42	15.0	44.7	57.2	72.6	71.6	39.2	57.7	26.8	52.8
GDN-880M	11.37	**12.9**	**47.6**	57.3	73.3	71.4	38.7	**58.8**	28.6	53.7
Mamba-2-880M	11.35	13.8	45.0	58.1	72.5	72.3	38.7	56.8	**30.2**	53.4
Mamba-3-SISO-880M	**11.23**	**12.9**	47.2	**58.8**	**73.6**	**72.7**	**40.2**	58.4	30.0	**54.4**
Mamba-3-MIMO-880M	**11.11**	**11.8**	**49.5**	**59.2**	**73.7**	**74.7**	**41.2**	**59.9**	28.6	**55.3**
Transformer-1.5B	10.51	11.1	**50.3**	60.6	73.8	74.0	40.4	58.7	29.6	55.4
GDN-1.5B	10.45	**10.9**	49.2	61.3	**74.3**	75.3	41.2	58.0	31.6	55.8
Mamba-2-1.5B	10.47	12.0	47.8	61.4	73.6	75.3	41.8	57.5	**32.6**	55.7
Mamba-3-SISO-1.5B	**10.35**	**10.9**	49.4	**61.9**	73.6	**75.9**	**42.7**	**59.4**	32.0	**56.4**
Mamba-3-MIMO-1.5B	**10.24**	**10.2**	**51.7**	**62.3**	**75.3**	**76.5**	**44.5**	**60.6**	**32.6**	**57.6**

Downstream Language Modeling Evaluations for Pretrained Models.

We find that our new Mamba-3 model *outperforms* the prior Mamba-2 model and strong linear attention alternatives, such as GDN, on language modeling across various pretrained model scales. **Mamba-3-SISO** is directly comparable to prior linear models; for example, it matches Mamba-2 exactly in architecture shapes (model dimensions, state size, etc.) and has comparable training time. Our **MIMO** variant of Mamba-3 further boosts accuracy on our downstream tasks by more than 1 percentage point over the regular Mamba-3 at the 1B scale, with the caveat that MIMO requires longer training times but not longer decoding latencies!

▶ How can training costs go up but not inference?

Retrieval Tasks

	Model (1.5B)	SWDE	SQD.	FDA	TQA	NQ	Drop	NIAH-Single-1			NIAH-Single-2			NIAH-Single-3		
	Context Length	2048						1024	2048	4096	1024	2048	4096	1024	2048	4096
Pure	Transformer	48.9	46.6	58.4	67.5	31.7	26.4	100.0	100.0	0.0	92.2	100.0	0.0	98.6	99.4	0.0
Pure	GDN	**32.7**	40.0	**28.3**	63.5	25.7	24.5	**100.0**	**100.0**	**99.8**	**100.0**	93.8	49.8	83.8	68.4	**34.2**
Pure	Mamba-2	30.7	39.1	23.7	64.3	25.1	**28.5**	**100.0**	99.6	62.0	**100.0**	53.8	11.8	**95.8**	**87.4**	13.4
Pure	**Mamba-3 SISO**	28.5	**40.1**	23.4	**64.5**	**26.5**	27.4	**100.0**	**100.0**	88.2	**100.0**	**95.4**	**50.6**	92.4	81.4	**34.2**
Pure	**Mamba-3 MIMO**	**36.3**	**41.7**	**29.3**	**64.5**	26.2	26.3	**100.0**	**100.0**	93.0	**100.0**	86.0	40.4	**95.8**	84.4	25.6
Hybrid	GDN	54.6	**48.4**	58.8	64.9	32.7	**30.0**	**100.0**	**100.0**	71.4	99.6	**100.0**	60.2	70.0	96.2	24.0
Hybrid	Mamba-2	58.2	45.6	**71.0**	**66.1**	**33.4**	28.1	**100.0**	**100.0**	3.2	99.6	98.8	0.0	98.2	98.0	0.0
Hybrid	Mamba-3 SISO	58.5	47.0	65.9	64.8	**33.4**	27.0	**100.0**	**100.0**	36.2	**100.0**	**100.0**	9.4	**99.8**	**100.0**	8.8
Hybrid	Mamba-3 SISO Norm*	**58.6**	47.3	52.4	65.7	33.3	28.5	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**96.0**	**99.8**	97.2	**56.8**

Real-world and Synthetic Retrieval Tasks.

Linear models, with their fixed-size state, **naturally underperform** their Transformer counterparts on **retrieval-based tasks**. As expected, within pure models, the Transformer is superior on retrieval tasks, but Mamba-3 performs well within the class of sub-quadratic alternatives. Interestingly, the addition of MIMO further improves retrieval performance *without increasing the state size*.

Given this innate deficit but overall strong modeling performance,

> *we predict that linear layers will be predominantly used in **conjunction** with global self-attention layers in the future.* *

* at least for language modeling

Hybrid models that combine the general *memory-like* nature of linear layers with the exact *database-like* storage of self-attention's KV cache have been shown empirically to outperform pure models while enabling significant memory and compute savings [9], and we do find here that the combination of linear layers with self-attention enables better retrieval compared to a vanilla Transformer.

However, we highlight that the exact way that these linear models interact with self-attention is *not fully understood*. For instance, we find that the use of the optional pre-output projection for Mamba-3 improves the length generalization performance on the synthetic NIAH tasks at the slight cost of in-context real-world retrieval tasks. Furthermore, even the details of the returned norm such as placement, e.g., pre-gate vs post-gate, and type, grouped vs regular, have non-negligible effects on accuracy on tasks composed of semi-structured and unstructured data, such as FDA and SWDE.

Kernels Here, There, and Everywhere

We're excited to see what people build with Mamba-3. To help facilitate this, we are open-sourcing our kernels, which are **on par in terms of speed** with the original Mamba-2 Triton kernels.

Benchmarking Latencies

Prefill Latency

Model	n=512	1024	2048	4096	16384
vLLM (Llama-3.2-1B)	**0.26**	**0.52**	**1.08**	**2.08**	**12.17**
Gated DeltaNet	0.51	1.01	2.01	4.00	16.21
Mamba-2	0.51	1.02	2.02	4.02	16.22
Mamba-3 (SISO)	0.51	1.01	2.02	4.01	16.22
Mamba-3 (MIMO $R = 4$)	0.60	1.21	2.42	4.76	19.44

Prefill+Decode Latency

Model	n=512	1024	2048	4096	16384
vLLM (Llama-3.2-1B)	4.45	9.60	20.37	58.64	976.50
Gated DeltaNet	4.56	9.11	18.22	36.41	145.87
Mamba-2	4.66	9.32	18.62	37.22	149.02
Mamba-3 (SISO)	**4.39**	**8.78**	**17.57**	**35.11**	**140.61**
Mamba-3 (MIMO $R = 4$)	4.74	9.48	18.96	37.85	151.81

Prefill and prefill+decode (same token count for both prefill and decode) latencies across sequence lengths for a 1.5B model on a single H100-SXM 80GB GPU. A batch size of 128 was used for all sequence lengths, wall-clock times (in seconds) are reported over three repetitions.

When comparing models at the 1.5B scale, Mamba-3 (SISO variant) *achieves the fastest prefill + decode latency* across all sequence lengths, outperforming Mamba-2, Gated DeltaNet, and even the Transformer with its highly optimized vLLM ecosystem. Furthermore, **Mamba-3 MIMO is comparable to Mamba-2 in terms of speed but has much stronger performance**.

Mamba-3 SISO's Triton-based prefill maintains nearly identical performance to Mamba-2, demonstrating that the new discretization and data-dependent RoPE embeddings do not introduce additional overhead, while Mamba-3 MIMO only incurs a moderate slowdown for prefill due to its efficient TileLang implementation. The strong decode performance for both Mamba-3 variants can be partially attributed to the CuTe DSL implementation, which was made significantly easier by the simplicity of Mamba-3 components.

Design Choices

We spent a lot of time thinking about how to make the kernels as fast as possible without compromising on ease-of-use. We ended up using the following stack: **Triton**, **TileLang**, and **CuTe DSL**.

The use of **Triton** was quite an easy choice. It's pretty much standard for architecture development (the great flash linear attention repo is purely in PyTorch and Triton) for good reason, as it enables better performance than standard PyTorch by enabling controlled tiling and kernel fusion while being a platform-agnostic language. Triton also has some pretty nifty features, like PTX (a GPU-oriented assembly language) injection and its Tensor Memory Accelerator support (on Hopper GPUs) for bulk, asynchronous transfers from global to shared memory.

Our MIMO prefill kernels were developed with **TileLang** instead. The additional projections corresponding with the variant present an opportunity where we can reduce memory IO via strategic manipulation across a GPU's memory hierarchy. Unfortunately, Triton didn't provide the granularity of memory control we desired, so we opted for TileLang, which allows us to explicitly declare and control shared-memory tiles and create register fragments, reusing memory more efficiently while still being high-level enough for us to develop the kernels quickly.

Since we've been hammering the importance of inference and decode, we decided to use **CuTe DSL** for our decode kernels. Through its Python interface, we're able to generate low-level kernels using high-level abstractions from CUTLASS. Here, we practically have CUDA-level control, enabling us to develop highly-performant kernels tailored to the specifications of our hardware (Hopper GPUs, in this case). With fine-grained control over tensor layouts and warp specialization, we built a kernel that takes advantage of all the bells and whistles in the GPU.

Importantly, these implementations across varying levels of GPU abstraction are made possible by the **underlying algorithmic design** of Mamba-3's simple, lightweight additions and their clever instantiations. We discuss details such as the exact fusion structure and kernel DSL in more depth in our full release.

Next Up

Glad you made it to the end of Part 1! There were a lot of details regarding our kernels and experimental results and ablations we didn't have time to cover in this post, but don't fret! Everything can be found in our paper, and the kernels have been open-sourced at mamba-ssm!

Up next, the second (and final) part of the series delves into the three core improvements to Mamba-3 and their SSM foundations, and gives some directions we're especially interested in.

Footnotes

1. or "BCNorm" in SSM terminology [↩]

References

1. **Mamba: Linear-Time Sequence Modeling with Selective State Spaces** [PDF]
   Gu, A. and Dao, T., 2024.
2. **Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality** [PDF]
   Dao, T. and Gu, A., 2024.
3. **Gated Delta Networks: Improving Mamba2 with Delta Rule** [PDF]
   Yang, S., Kautz, J. and Hatamizadeh, A., 2025.
4. **Learning to (Learn at Test Time): RNNs with Expressive Hidden States** [PDF]
   Sun, Y., Li, X., Dalal, K., Xu, J., Vikram, A., Zhang, G., Dubois, Y., Chen, X., Wang, X., Koyejo, S., Hashimoto, T. and Guestrin, C., 2025.
5. **Hungry Hungry Hippos: Towards Language Modeling with State Space Models** [PDF]
   Fu, D.Y., Dao, T., Saab, K.K., Thomas, A.W., Rudra, A. and Ré, C., 2023.

6. **In-context Learning and Induction Heads**
Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S. and Olah, C., 2022. Transformer Circuits Thread.

7. **RWKV: Reinventing RNNs for the Transformer Era** [PDF]
Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K.K., He, X., Hou, H., Lin, J., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Mantri, K.S.I., Mom, F., Saito, A., Song, G., Tang, X., Wang, B., Wind, J.S., Wozniak, S., Zhang, R., Zhang, Z., Zhao, Q., Zhou, P., Zhou, Q., Zhu, J. and Zhu, R., 2023.

8. **Test-time regression: a unifying framework for designing sequence models with associative memory** [PDF]
Wang, K.A., Shi, J. and Fox, E.B., 2025.

9. **An Empirical Study of Mamba-based Language Models** [PDF]
Waleffe, R., Byeon, W., Riach, D., Norick, B., Korthikanti, V., Dao, T., Gu, A., Hatamizadeh, A., Singh, S., Narayanan, D., Kulshreshtha, G., Singh, V., Casper, J., Kautz, J., Shoeybi, M. and Catanzaro, B., 2024.

© Copyright 2026 Goomba AI Lab.